

Zmanim RSS Parsing Strategy (Didan Project)

Summary:

The Chabad zmanim RSS feed is not a stable API. Labels vary across days, holidays, and special cases. A deterministic lookup approach (exact string matching) is insufficient. Instead, we must classify labels semantically and preserve unknown data.

Key Problems Identified:

- Label variants for the same zman (e.g., multiple forms of tzeis)
- Composite labels (e.g., 'Sunset (Shkiah)Fast Begins')
- Special-event-only labels (Chametz deadlines, etc.)
- No guarantee of completeness across calendar cycles

Proposed Solution:

1. Normalize malformed labels
2. Split composite labels
3. Classify each component
4. Map to canonical zmanim OR store as dynamic events

Normalization Function:

```
func normalizeLabel(s string) string {
    s = strings.ReplaceAll(s, ")Fast", ") | Fast")
    return s
}
```

Splitting Function:

```
func splitLabel(s string) []string {
    s = normalizeLabel(s)
    parts := strings.Split(s, "|")
    for i := range parts {
        parts[i] = strings.TrimSpace(parts[i])
    }
    return parts
}
```

Classifier Rules:

```
func classify(lbl string) (fields []string, isEvent bool) {
    switch {
    case strings.Contains(lbl, "Alot Hashachar"):
        return []string{"alos"}, false
    case strings.Contains(lbl, "Misheyakir"):
        return []string{"misheyakir"}, false
    case strings.Contains(lbl, "Sunrise"):
        return []string{"sunrise"}, false
    case strings.Contains(lbl, "Sunset"):
        return []string{"shkiah"}, false
    case strings.Contains(lbl, "Tzeit Hakoachavim"),
        strings.Contains(lbl, "Shabbat Ends"),
        strings.Contains(lbl, "Holiday Ends"),
        strings.Contains(lbl, "Fast Ends"),
        strings.Contains(lbl, "Holiday/Fast Ends"):
        return []string{"tzeis"}, false
    case strings.Contains(lbl, "Candle Lighting"):
        return []string{"tzeis"}, true
    }
    return nil, true
}
```

```
}
```

Integration:

```
labels := splitLabel(prefix)
for _, lbl := range labels {
    fields, isEvent := classify(lbl)

    for _, f := range fields {
        setField(z, f, t)
    }

    if isEvent || len(fields) == 0 {
        if z.Events == nil {
            z.Events = make(map[string]time.Time)
        }
        z.Events[lbl] = t
    }
}
```